

ACEED — Automatización en la Calificación de Evaluaciones Escritas Digitalizadas

ASP.NET MVC · .NET Framework 4.8 · Google OAuth/Drive · Gemini Flash

1. Resumen Ejecutivo

Aplicación web en **ASP.NET MVC sobre .NET Framework 4.8** que automatiza la corrección de exámenes escritos a mano. El flujo real del docente es el siguiente:

1. El docente posee el **examen con respuestas correctas en PDF** (100% legible, es el documento base que usa para imprimir).
2. Imprime ese mismo examen **sin las respuestas** y lo reparte a los estudiantes.
3. Los estudiantes resuelven el examen **a mano** y lo entregan.
4. El docente **escanea cada examen individualmente**: 1 PDF = 1 estudiante.
5. Sube al sistema: (a) el examen base con respuestas, una sola vez por "apartado de examen", y (b) los PDFs escaneados de los estudiantes correspondientes a ese examen base.
6. El sistema compara, califica y entrega la **nota de forma inmediata**, con **feedback explicativo** en los casos donde la IA no tiene certeza total (caligrafía ambigua, paráfrasis, respuestas parciales).

Restricciones y reglas de negocio confirmadas:

Regla	Detalle
Tamaño máximo de examen	10 hojas (aplica tanto al examen base como al del estudiante, son estructuralmente idénticos)
Formato de entrada	PDF en ambos casos (clave y entregas)
Relación PDF–alumno	1 PDF = 1 alumno (el docente escanea examen por examen)
Examen base	Siempre 100% legible (documento digital de origen)
Examen del alumno	Enunciado impreso legible; respuestas manuscritas con caligrafía variable
Apartados de examen	Un docente maneja múltiples exámenes base; cada lote de entregas se asocia a su examen base correspondiente

Regla	Detalle
Jerarquía académica	Curso → Unidad → Tipo de Evaluación → Examen Base. El docente crea el curso, dentro del curso sus unidades, y dentro de cada unidad el tipo de evaluación (examen teórico, parcial, etc.) al que pertenece el examen base
Escala de calificación	Configurable por examen base. Por defecto, escala vigesimal peruana (0–20), pero el docente puede definir cualquier nota máxima (p. ej. 10, si el examen teórico vale la mitad y el práctico la otra mitad). El sistema normaliza los puntajes de las preguntas a la escala elegida
Motor de IA	Gemini Flash (API REST, visión multimodal)

Principio de diseño central: ningún motor de reconocimiento de manuscrito es infalible. La robustez del sistema no proviene de un OCR perfecto, sino de la arquitectura: *evaluación con niveles de confianza + comparación semántica (no literal) + feedback explicativo + bandeja de revisión humana solo para casos dudosos*. El docente interviene por excepción (típicamente 5–10% de las respuestas), no revisa todo.

2. Alcance

2.1 Incluido (MVP)

- Login con Google OAuth 2.0 y permiso de Google Drive del usuario.
- Gestión de la **estructura académica**: Cursos → Unidades → Tipos de Evaluación.
- Gestión de **Exámenes Base** dentro de cada tipo de evaluación: subir PDF con respuestas, definición de la **nota máxima** (vigesimal 0–20 por defecto, configurable: 10, 100, etc.), calibración de la clave, activación.
- Subida de entregas de alumnos (individual o en lote: N archivos PDF = N alumnos).
- Pipeline asíncrono de corrección: rasterización → evaluación con Gemini Flash → consolidación de nota.
- **Sistema de feedback por respuesta**: transcripción, veredicto, justificación y nivel de confianza.
- Bandeja de revisión para respuestas de baja confianza (decisión del docente en un clic).
- Publicación de notas y exportación de reportes (Excel/PDF).
- Almacenamiento de todos los archivos en el Google Drive del docente.

2.2 Excluido (fuera del MVP)

- Separación automática de múltiples alumnos dentro de un mismo PDF (regla: 1 PDF = 1 alumno).
- Entrenamiento de modelos propios de HTR/OCR.
- Portal para estudiantes (el sistema es de uso exclusivo del docente en el MVP; puede ser trabajo futuro).
- Exámenes con contenido no textual complejo como respuesta (dibujos, planos, partituras). Fórmulas matemáticas simples sí están dentro del alcance.

3. Stack Tecnológico

Capa	Tecnología	Notas
Web	ASP.NET MVC 5, C#, .NET Framework 4.8, IIS	Requisito del proyecto. Solución <code>grpSPEF/grpSPEF.sln</code> , proyecto <code>pjtSPEF</code>
Autenticación	Google OAuth 2.0 vía OWIN (<code>Microsoft.Owin.Security.Google</code>)	<i>Pendiente de credenciales.</i> Mientras tanto: Forms Authentication en modo desarrollo detrás de la interfaz <code>ICurrentUserService</code> (usuario <code>dev@spef.local</code>); migrar a OAuth no toca los controladores
Almacenamiento de archivos	Google Drive API v3 (<code>Google.Apis.Drive.v3</code>)	<i>Pendiente de credenciales.</i> Mientras tanto: disco local (<code>App_Data/storage</code>) detrás de la interfaz <code>IFileStorageService</code> ; el modelo usa una referencia genérica <code>archivo_ref</code> que mañana será el <code>DriveFileId</code>
Base de datos	SQL Server + Entity Framework 6	La BD se crea con scripts manuales en <code>database/</code> (ver §5); EF6 corre con initializer deshabilitado y mapeo explícito a tablas <code>snake_case</code>
Validación de PDF	PdfPig 0.1.14	Verifica que el archivo sea un PDF real y cuente ≤ 10 páginas
Rasterización PDF	PDFium (PdfiumViewer) o Ghostscript	<i>Futuro.</i> PDF $\rightarrow$ imagen 300 DPI (calidad crítica para lectura de manuscrito)
Extracción de texto de la clave	PdfPig	<i>Futuro.</i> Texto nativo del PDF base (digital) — precisión total, sin OCR
IA de evaluación	Gemini Flash vía API REST	<i>Futuro.</i> Visión multimodal: transcribe y evalúa en un solo paso. Usar <code>responseMimeType: application/json</code> + <code>responseSchema</code> para salida estructurada garantizada
Jobs en background	Hangfire	<i>Futuro.</i> Compatible con .NET 4.8. Requiere configurar IIS con <i>Always Running</i> / sin idle timeout para que los lotes no mueran con el reciclaje del app pool
Cifrado de tokens	DPAPI / <code>MachineKey</code>	<i>Futuro.</i> El refresh token de Google se almacenará cifrado

4. Arquitectura de Componentes



Estructura de almacenamiento (hoy local en `App_Data/storage`, futuro espejo en Drive):

```

storage/
├── exámenes-base/{guid}.pdf ← PDF clave de cada examen base
└── entregas/{guid}.pdf      ← (futuro) 1 PDF por alumno
  
```

5. Modelo de Datos (núcleo)

La base de datos se crea ejecutando `database/script.sql` (consolidado, idempotente: crea la BD SPEF completa de una vez). Los scripts por objeto están en `database/tables/` y `database/seed/`, numerados en orden de ejecución. Cada incremento añade sus scripts y regenera el consolidado.

Entidad	Campos principales	Estado
<code>usuarios</code>	google_id (NULL hasta OAuth), email, nombre, refresh_token (NULL hasta OAuth, cifrado), activo	☒ Creada

Entidad	Campos principales	Estado
<i> cursos </i>	nombre, periodo, docente (FK usuario)	☑ Creada
<i> unidades </i>	curso (FK, cascade), número, nombre — UNIQUE(curso, número)	☑ Creada
<i> tipos_evaluacion </i>	unidad (FK, cascade), nombre (p. ej. "Examen teórico", "Práctica calificada")	☑ Creada
<i> examenes_base </i>	tipo de evaluación (FK, cascade), título, archivo_ref (genérico: ruta local hoy, DriveFileId mañana), archivo_nombre_original , total_paginas (CHECK 1–10), nota_maxima (DECIMAL, default 20, CHECK > 0), estado: <i>Borrador</i> → <i>Calibrado</i> → <i>Activo</i>	☑ Creada
<i> preguntas_clave </i>	examen base (FK), número, enunciado, respuesta correcta, tipo, puntaje (suma = nota_maxima), página, criterios de puntaje parcial	⌚ Próximo incremento
<i> entregas_alumno </i>	examen base (FK), alumno, archivo_ref, estado del procesamiento, nota final	⌚ Próximo incremento
<i> respuestas_evaluadas </i>	entrega (FK), pregunta (FK), transcripción, veredicto, puntajes, confianzas, feedback , requiere_revisión	⌚ Próximo incremento
<i> log_procesamiento </i>	entrega (FK), etapa, resultado, duración, error	⌚ Próximo incremento

Nota de diseño: la identificación del alumno puede capturarse de dos formas: (a) el docente la escribe al subir el PDF, o (b) Gemini transcribe el nombre manuscrito de la cabecera del examen y el docente solo lo confirma. Implementar (a) primero y (b) como mejora.

6. Pipeline de Corrección (detalle técnico)

6.1 Calibración del examen base (una vez por apartado)

1. Docente sube el PDF con respuestas dentro del tipo de evaluación correspondiente (Curso → Unidad → Tipo de Evaluación). ☑ *Implementado*
2. Define la **nota máxima del examen**: por defecto 20 (escala vigesimal), modificable a cualquier valor (10, 100, etc.). ☑ *Implementado*
3. PdfTextService extrae el texto nativo (el PDF es digital, precisión total). ⌚
4. Una llamada a Gemini Flash con el texto completo estructura la clave: lista de preguntas con {numero, enunciado, respuesta_correcta, puntaje_sugerido, tipo, pagina} en JSON. El sistema propone una distribución de puntajes proporcional a la nota máxima definida. ⌚

5. **Vista de confirmación:** el docente revisa la lista extraída, corrige enunciados/respuestas/puntajes si hace falta, ajusta el puntaje real de cada pregunta y los criterios de puntaje parcial si los hay.
Validación obligatoria: la suma de puntajes de las preguntas debe igualar la nota máxima (el sistema lo muestra en tiempo real, p. ej. "18.0 / 20.0 asignados"). ⌚
6. El examen pasa a estado **Activo** y queda listo para recibir entregas. ⌚

Esta vista de confirmación es la garantía de calidad de la mitad del problema: la clave queda validada por el humano antes de calificar a nadie.

6.2 Procesamiento de cada entrega (job Hangfire) ⌚

1. **Validación de entrada:** PDF $\leq$ 10 páginas, resolución suficiente (advertir si el escaneo es < 200 DPI efectivos).
2. **Rasterización:** cada página $\rightarrow$ imagen 300 DPI (JPEG calidad ~ 85 : PNG a 300 DPI puede exceder el límite de ~ 20 MB por request de Gemini).
3. **Evaluación con Gemini Flash** enviando **todas las páginas del examen en una sola llamada** con la clave completa (≤ 10 páginas lo permite y evita el problema de respuestas que continúan en otra página). El prompt incluye:
 - Las preguntas con su enunciado, respuesta correcta, puntaje y criterios (texto ya validado de la clave).
 - Las imágenes de las páginas del alumno.
 - `responseSchema` JSON para salida estructurada garantizada.
4. **Consolidación** (`ScoringService`): suma de puntajes, aplicación de umbrales de confianza, marcado de respuestas para revisión.
5. **Resultado:** nota preliminar inmediata + detalle por pregunta con feedback.

6.3 Contrato JSON de salida de Gemini (por pregunta)

```
{
  "numero": 3,
  "transcripcion": "La fotosíntesis es el proceso por el cual las plantas
convierten luz en energía química",
  "veredicto": "correcta",
  "puntaje_sugerido": 2.0,
  "puntaje_maximo": 2.0,
  "confianza_lectura": 0.93,
  "confianza_evaluacion": 0.97,
  "feedback": "La respuesta es conceptualmente correcta aunque está
parafraseada respecto a la clave y contiene errores ortográficos ('proceso'),
que no afectan el contenido. Coincide en los elementos esenciales: conversión
de luz en energía química por las plantas."
}
```

6.4 Manejo de paráfrasis y respuestas no literales — el sistema de feedback

Este es el corazón de la flexibilidad solicitada. La comparación **nunca es literal** (no se usa matching de strings ni distancia de edición como criterio principal): Gemini evalúa **equivalencia semántica** entre lo

que escribió el alumno y la respuesta correcta, usando el enunciado como contexto. Las instrucciones del prompt del evaluador cubren explícitamente estos casos:

Caso	Comportamiento del sistema
Paráfrasis ("las plantas transforman la luz solar en alimento" vs. la clave textual)	Veredicto correcta si los conceptos esenciales coinciden. El feedback explica qué elementos de la clave están presentes
Equivalencias formales ("2π" vs "6.28", "H ₂ O" vs "agua")	Correcta, con nota en el feedback de la forma equivalente usada
Errores ortográficos que no alteran el significado	No penalizan (configurable por el docente por examen)
Respuesta parcial (menciona 2 de 3 elementos pedidos)	Veredicto parcial , puntaje proporcional según criterios de la clave, feedback indicando qué faltó
Caligrafía ambigua (confianza_lectura baja)	Veredicto provisional + requiere_revision = true . El feedback describe la ambigüedad: <i>"La palabra clave podría leerse como 'mitosis' o 'meiosis'; el resto de la respuesta sugiere mitosis"</i>
Equivalencia dudosa (confianza_evaluacion baja: la paráfrasis es lejana o incompleta)	requiere_revision = true con feedback argumentando ambas lecturas posibles, para que el docente decida con contexto
Ilegible o en blanco	Veredicto ilegible/incorrecta según el caso, siempre a revisión si es ilegible

Reglas de decisión del **ScoringService** (umbrales configurables):

- **confianza_lectura** ≥ 0.85 y **confianza_evaluacion** ≥ 0.85 → puntaje automático.
- Cualquiera de las dos por debajo del umbral → **bandeja de revisión**: el docente ve la imagen de la respuesta, la transcripción, el feedback de la IA y aprueba/corriges el puntaje en un clic.
- Preguntas de tipo *desarrollo largo* → siempre pasan por revisión al inicio (auto-calificación solo para opción múltiple y respuesta corta; se relaja según resultados del piloto).
- Importante: las confianzas auto-reportadas por LLMs tienden a agruparse alto (0.9+) aunque se equivoquen; los umbrales deben **calibrarse con datos reales desde las primeras pruebas**, no asumirse.

El **feedback se conserva siempre**, incluso en respuestas auto-calificadas: es el registro de *por qué* el sistema asignó ese puntaje, sirve como justificación ante reclamos de estudiantes y es evidencia documental para la validación del sistema.

6.5 Esqueleto del prompt del evaluador

Eres un asistente de corrección de exámenes universitarios. Recibes la imagen de una página de examen resuelta a mano por un estudiante y la clave de respuestas de esa página.

CLAVE DE LA PÁGINA:

[Pregunta 3 | Puntaje máx: 2.0 | Tipo: respuesta corta]

Enunciado: ¿Qué es la fotosíntesis?

Respuesta correcta: Proceso mediante el cual las plantas convierten la energía lumínica en energía química.

Criterios de puntaje parcial: 1.0 si menciona conversión de energía sin especificar tipos.

INSTRUCCIONES:

1. Localiza cada pregunta en la imagen y transcribe fielmente la respuesta manuscrita del estudiante, sin corregirla.
2. Evalúa EQUIVALENCIA SEMÁNTICA con la respuesta correcta. Acepta paráfrasis, sinónimos y formas equivalentes. Los errores ortográficos no penalizan si no alteran el significado.
3. Asigna puntaje según los criterios. Usa "parcial" cuando corresponda.
4. Reporta confianza_lectura (qué tan seguro estás de haber leído bien la caligrafía) y confianza_evaluacion (qué tan seguro estás del veredicto) como valores entre 0 y 1. Sé conservador: ante ambigüedad real, reporta confianza baja y explica la ambigüedad en el feedback.
5. El feedback debe justificar el puntaje en 1-3 oraciones, en español, dirigido al docente.
6. Responde ÚNICAMENTE con el arreglo JSON, sin texto adicional ni markdown.

7. Riesgos y Mitigaciones

#	Riesgo	Impacto	Mitigación
1	Calidad del escaneo (el riesgo #1, mayor que la caligrafía: a baja resolución todo motor falla)	Alto	Validar resolución al subir, advertencias inmediatas, guía de escaneo para el docente, preprocesamiento (deskew, contraste)
2	Caligrafía extremadamente difícil	Medio	Diseñado en la arquitectura: confianza baja → revisión humana con feedback explicativo. El sistema nunca "adivina en silencio"
3	Respuestas de desarrollo largo	Medio	Siempre a revisión al inicio; el feedback de la IA acelera la decisión del docente. Auto-calificación gradual según resultados del piloto
4	Verificación OAuth de Google	Medio	Scope drive.file (restringido) en lugar del scope completo de Drive → proceso de verificación leve, sin auditoría costosa
5	Cambios/límites de la API de Gemini	Medio	Capa GeminiGradingService aislada tras interfaz → el motor es intercambiable. Reintentos con backoff y rate limiting

#	Riesgo	Impacto	Mitigación
6	Costos de API	Bajo	Gemini Flash cuesta centavos por examen de 10 páginas. Registrar costo por examen como métrica del piloto
7	Examen base mal estructurado (numeración irregular, formato atípico)	Medio	La vista de confirmación de la calibración obliga validación humana de la clave antes de activar el examen
8	Alucinación del modelo (transcribir algo que no está)	Medio	Prompt conservador (instrucción explícita de reportar baja confianza ante ambigüedad), umbrales estrictos al inicio, feedback siempre auditable junto a la imagen original
9	Hangfire sobre IIS: reciclaje del app pool mata los jobs	Medio	Configurar el app pool con <i>Always Running</i> , deshabilitar idle timeout y habilitar auto-start del sitio

8. Métricas de Éxito (para el piloto)

- **Concordancia** con la corrección manual del docente (objetivo: $\geq 90\%$ en respuesta corta y opción múltiple).
- **% de auto-calificación** (objetivo: $\geq 85\%$ de respuestas sin intervención manual).
- **Tiempo de corrección por examen** vs. corrección manual (objetivo: reducción $\geq 90\%$ del tiempo del docente).
- **Costo por examen** (objetivo documentado: $< \text{USD } 0.05$ por examen de 10 páginas con Gemini Flash).
- **Tasa de reclamos sostenidos**: respuestas auto-calificadas que el docente revierte tras reclamo del estudiante (objetivo: $< 2\%$).

9. Decisiones de Diseño Deliberadas (qué NO se hará y por qué)

1. **No entrenar un modelo propio de HTR/OCR**: meses de trabajo, dataset enorme requerido y resultado inferior a los modelos multimodales actuales.
2. **No comparar respuestas por matching de strings ni distancia Levenshtein** como criterio principal: se rompe con sinónimos, paráfrasis y errores ortográficos. La equivalencia semántica vía LLM es el mecanismo central.
3. **No usar transcripción y evaluación en dos pasos separados**: Gemini transcribe y evalúa en una sola llamada viendo la respuesta en contexto del enunciado, lo que mejora la lectura de caligrafía ambigua.
4. **No prometer "corrección 100% automática sin intervención"**: la promesa correcta y defendible es "*reducción superior al 90% del tiempo de corrección, con supervisión por excepción y feedback auditable en cada respuesta*". Es medible, honesta y técnicamente sostenible.
5. **No aceptar PDFs con múltiples alumnos**: la regla 1 PDF = 1 alumno (máx. 10 hojas) elimina el problema complejo de segmentar exámenes dentro de un archivo y simplifica todo el pipeline.

10. Estado de Avance

Cómo correr el proyecto

1. Ejecutar `database/script.sql` (SSMS, o `sqlcmd -S "(localdb)\MSSQLLocalDB" -i database/script.sql -b -I`). Crea la BD **SPEF** completa; es idempotente.
2. Si no se usa LocalDB, ajustar el connection string `SpefDb` en `grpSPEF/pjtSPEF/Web.config`.
3. Abrir `grpSPEF/grpSPEF.sln` en Visual Studio 2022 y ejecutar (F5), o compilar con MSBuild y servir con IIS Express.
4. Entrar con el botón **"Entrar (modo desarrollo)"** (usuario `dev@spef.local`, sin contraseña — auth simulada hasta tener credenciales de Google).

Implementado hasta hoy

- ☒ Estructura del proyecto (MVC 5, .NET 4.8) con capas: `Models/Entities`, `Models/ViewModels`, `Data` (EF6 + mapeos `snake_case`), `Services` (interfaces + implementaciones).
- ☒ Esquema SQL inicial en `database/`: usuarios, cursos, unidades, tipos_evaluacion, examenes_base (+ seed del usuario dev).
- ☒ Autenticación de desarrollo (Forms Auth) detrás de `ICurrentUserService`; toda la app exige sesión (`AuthorizeAttribute` global) y autoriza **por recurso** (cada consulta navega hasta el dueño).
- ☒ CRUD jerárquico completo con navegación drill-down y breadcrumbs: Cursos → Unidades → Tipos de Evaluación → Exámenes Base.
- ☒ Subida de PDF del examen base con validación real (magic bytes + conteo de páginas ≤ 10 con PdfPig), almacenamiento local tras `IFileStorageService`, descarga, reemplazo y borrado con limpieza de archivos (incluida la cascada al borrar curso/unidad/tipo).
- ☒ Nota máxima configurable por examen (default 20) y estados Borrador/Calibrado/Activo (por ahora todo queda en Borrador).

Pendiente (próximos incrementos)

- ⌚ Calibración de la clave: extracción de texto (PdfPig) + estructuración con Gemini + vista de confirmación/edición de preguntas y puntajes.
- ⌚ Entregas de alumnos, pipeline de corrección (Hangfire + rasterización + Gemini), bandeja de revisión, reportes.
- ⌚ Integraciones reales: Google OAuth (OWIN), Google Drive, API key de Gemini.

Registro de cambios

Fecha	Incremento	Cambios
2026-06-11	1	Plantilla limpia y rebrandeada, <code>.gitignore</code> , esquema SQL inicial (<code>database/</code>), EF6 + PdfPig instalados, auth dev (Forms) tras <code>ICurrentUserService</code> , CRUD jerárquico Cursos→Unidades→Tipos→Exámenes con subida/validación/descarga de PDF

Documento de diseño vivo: se actualiza con cada incremento.